DOCUMENTAÇÃO DO PROJETO / CATEGORIA NEXUS

ZIPPY

Guardião da Smart Home 2050

Desenvolvimento e demonstração de um simulador de robô doméstico, com navegação autônoma virtual, painel de controle e arquitetura modular.

EXPO TECH 2026.2 • MISSÃO 2050
NEXUS — Núcleo de Tecnologias e Engenharias
6º semestre • Ciência da Computação • UniFECAF
Evento: 28 de novembro de 2026

ROBÓTICA • PERCEPÇÃO • SOFTWARE • CIBERSEGURANÇA
Relatório técnico de desenvolvimento e validação

01 / MISSÃO E CONCEITO

Zippy — guardião virtual da casa

O projeto explora a aplicação de robótica móvel e automação residencial por meio de um ambiente virtual programável.

O Zippy é um simulador executável em Python e Pygame para navegação de um robô móvel em uma planta residencial. Ele combina cinemática diferencial, percepção virtual, planejamento de rota, controle e visualização em uma aplicação desktop.

Objetivos demonstrados

Eixo	Aplicação no Zippy
Robótica e sistemas embarcados	Modelo de robô diferencial, comandos de roda, odometria e ciclo de controle simulados.
Percepção e reação	Leituras virtuais de distância alimentam setores de percepção, mapa e decisões de desvio.
Smart Home 2050	Casa residencial representada por planta, cômodos, mobiliário e missões de patrulha/exploração.
Software Architecture	Componentes separados por responsabilidade, barramento publish/subscribe e máquina de estados.
Cibersegurança	Análise de caminhos de entrada inseguros e testes controlados reproduzíveis no software.

A navegação, as leituras, a pose e os estados apresentados neste relatório são gerados pelo simulador Zippy.

Tecnologias em execução

Python • Pygame • NumPy. O Blender compõe o material visual residencial e o modelo conceitual do robô.

02 / NAVEGAÇÃO AUTÔNOMA

Percepção, planejamento e reação

O ciclo autônomo conecta leituras simuladas a decisões de rota e comandos de movimento.

Etapa	Comportamento implementado
Percepção	LiDAR virtual por raycasting, 360 raios, ruído opcional e resumo por setores.
Mapeamento	Grade de ocupação atualizada por leituras, com estimativa de cobertura explorada.
Planejamento	Busca A* em grade; obstáculos inflados pelo raio do robô e margem; suavização e replanejamento quando o caminho bloqueia.
Controle	Máquina de estados, seguimento de caminho, ajuste de velocidade e desvio reativo diante de obstáculo.
Missões	Destino por clique/cômodo, patrulha, exploração por fronteiras e limpeza simulada.

Demonstração de navegação

Na execução validada, com ruído desligado, o Zippy recebeu a missão de ir à Cozinha e alcançou o objetivo em 5,3 segundos simulados (318 passos de atualização).

O controlador e a física operam sobre rodas, posição, distâncias e obstáculos virtuais; os resultados descrevem o cenário simulado.

Controles da demonstração

A interface oferece seleção de cômodos, patrulha, exploração, modo manual, pausa, obstáculo surpresa e alternância de visualizações. Os atalhos aparecem na barra inferior e no painel lateral.

03 / INTERFACE IMERSIVA

Painel de controle do simulador

A interface Pygame apresenta, durante a execução, a posição estimada e o estado interno da missão.

Área do painel	Informação exibida
Estado e missão	Modo operacional simulado e estado atual da máquina de estados.
Pose e movimento	Posição, orientação, odometria, erro estimado, velocidade e comandos de roda.
Percepção	Distâncias por setor do LiDAR e indicação visual de proximidade.
Mapa e rota	Planta, mapa aprendido, caminho planejado, waypoints e cobertura.
Tópicos locais	Frequência e atualização de mensagens como /scan, /odom, /path, /cmd_vel e /zippy/status.
Eventos	Registro das ações e reações da missão simulada.

As posições, sensores, odometria e mensagens mostradas no painel são telemetria interna produzida pelo ciclo de simulação.

Como apresentar

Inicie uma missão para um cômodo, acompanhe a rota e o indicador de estado, mostre os setores do LiDAR e compare a posição simulada com a odometria. A visualização pode alternar entre planta, mapa aprendido e grade usada no planejamento.

04 / SEGURANÇA OFENSIVA

Análise de software e testes controlados

Avaliação autorizada dos fluxos locais do programa, com registro de achados e resultados reproduzíveis.

Achado	Descrição técnica	Referência no arquivo de laboratório
VULN-01 • HIGH • CWE-502	O carregador de progresso encaminha o arquivo de save a pickle.load(), mecanismo que desserializa objetos Python.	Projeto_Zippy_vulneravel.py: load_progress(), linhas 1099 e 1129; atalho F9, linhas 1426–1427.
VULN-02 • HIGH • CWE-94	O carregador de cenário encaminha o conteúdo de cenario.cfg a eval() durante a inicialização.	Projeto_Zippy_vulneravel.py: load_scenario(), linhas 214 e 237; chamada no reset, linha 1007.

Verificações dinâmicas controladas

Caso	Resultado
Cenário benigno válido	A pose esperada foi carregada.
Cenário malformado	A entrada com sintaxe incompleta foi rejeitada pelo fluxo existente.
Caminho de eval	Um stub registrou a chamada e interrompeu a avaliação da entrada de teste.
Caminho de pickle	Um stub registrou o fluxo e retornou dados fixos; nenhum arquivo foi desserializado.
Versão principal	O cenário de teste foi ignorado; as rotas de carregamento inseguro não estão presentes.

Mitigações observadas

Na versão principal Projeto_Zippy.py, não aparecem os carregadores de cenário com eval() nem de save com pickle.load(). As verificações controladas confirmam a ausência dessas rotas no fluxo de inicialização usado nos testes.

Comando de reprodução: python -m unittest discover -s tests -v • Resultado observado: 5 testes aprovados.

05 / ARQUITETURA DE CÓDIGO

Componentes, padrões e justificativas

Organização lógica modular executada localmente em um único processo Python.

INTERFACE
Pygame: entradas, painel e visualizações

↕ mensagens locais

BUS
Publish/subscribe em memória

↕ tópicos internos

MISSÃO • BRAIN • PLANNER • MAPPER • PERCEPTION • LIDAR • ODOM • MOTOR
Destinos → rota → controle → rodas virtuais; leituras → mapa → replanejamento

↕ estado da simulação

SIM • WORLD • ROBOT
Geometria, pose, cinemática e colisões simuladas

Padrão / decisão	Aplicação e justificativa
Publish/subscribe	O Bus desacopla produtores e consumidores de sensores, mapas e comandos.
Nós periódicos	Separação por responsabilidade deixa explícitas as etapas do ciclo de atualização.
Máquina de estados (FSM)	BrainNode organiza modos de navegação e respostas a condições observadas.
A* e grade de ocupação	Separação entre mapa, planejamento global e controle facilita inspeção do percurso.

O barramento implementa publish/subscribe local e síncrono, inspirado no conceito de tópicos do ROS 2.

06 / VALIDAÇÃO DO PROJETO

Resultados funcionais observados

Ensaios locais documentados com Python 3.12.0rc3, Pygame 2.6.1 e NumPy 2.5.4.

Verificação	Resultado registrado
Sintaxe e importação	Compilação dos módulos e importação sem erro.
Inicialização da simulação	8 componentes instanciados; modo OCIOSO; estado PARADO.
LiDAR e mapeamento	Após 120 passos: scan de 360 raios e cobertura simulada aproximada de 29,7%.
Navegação para a Cozinha	Objetivo alcançado em 5,3 s simulados, em 318 passos, com ruído desligado.
Interface desktop	Inicialização com driver de tela dummy e carregamento da planta.
Testes de segurança controlados	5/5 testes passaram no runner unittest.

Execução

```
python -m pip install -r requirements.txt
python Projeto_Zippy.py
python -m unittest discover -s tests -v
```

Roteiro de apresentação

Passo	Demonstração
1	Abrir a planta e apresentar o robô e a interface.
2	Selecionar um cômodo e acompanhar o planejamento A* e a navegação.
3	Exibir LiDAR virtual, mapa, pose, odometria e estado da FSM.
4	Apresentar a arquitetura e explicar as decisões de organização do software.
5	Executar os testes controlados e apresentar os achados de segurança documentados.

ANEXO A / EVIDÊNCIAS VISUAIS

Modelagem residencial e conceito do robô

Material visual reproduzido a partir das imagens fornecidas junto às versões anteriores da documentação.



Figura A1. Modelo conceitual do robô.



Figura A2. Modelagem residencial — vista superior.



Figura A3. Ambiente residencial modelado no Blender.

Referências técnicas do projeto

Projeto_Zippy.py: arquitetura e navegação. Projeto_Zippy_vulneravel.py: fluxos usados na análise de segurança. tests/test_seguranca_dinamica.py: verificações controladas reproduzíveis. 13_NEXUS_6Sem_CDC_Guardiao.pdf: roteiro da categoria. primeira doc.pdf: material descritivo e imagens iniciais.

ANEXO B / INTEGRAÇÃO E AMBIENTE 3D

Visualização tridimensional do Zippy no Unity

O Unity foi incorporado ao projeto como ferramenta de integração e visualização tridimensional, permitindo apresentar o Zippy em um ambiente residencial virtual. Os modelos desenvolvidos no Blender são importados e organizados em um cenário que representa uma residência inteligente.

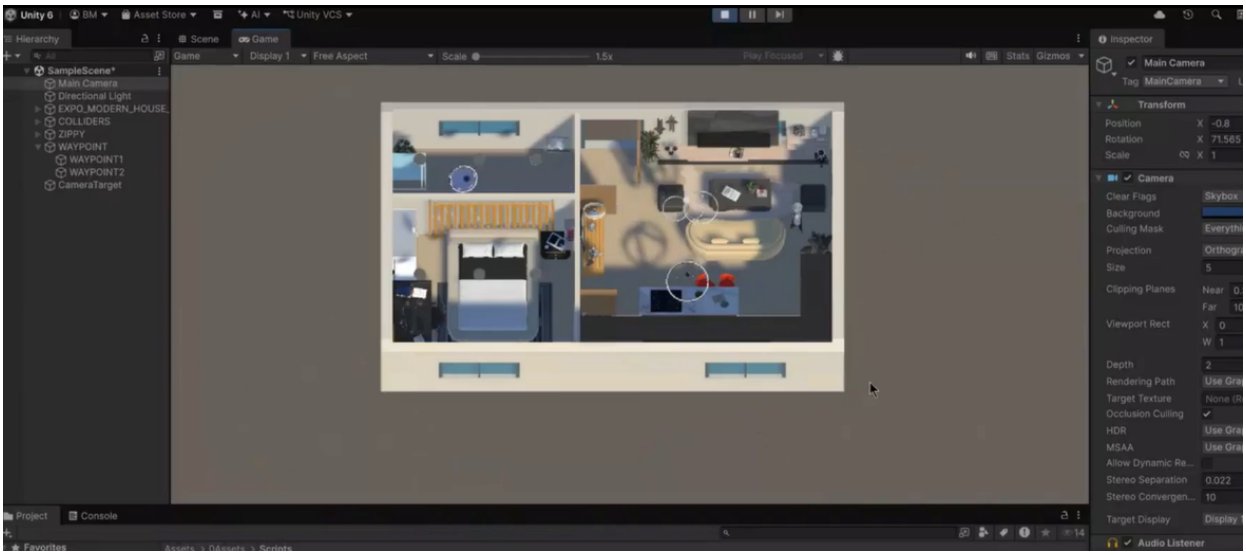


Figura 1. Ambiente residencial organizado no Unity.

Elementos desenvolvidos

Elemento	Descrição
Ambiente residencial	Construção e organização de um cenário virtual tridimensional que representa os espaços de uma Smart Home.
Modelo do robô	Importação e integração do modelo 3D do Zippy, desenvolvido no Blender.
Visualização	Organização da câmera, posicionamento dos objetos e apresentação do robô dentro do ambiente residencial.

A visualização tridimensional complementa a simulação funcional: o Unity apresenta os elementos e o cenário, enquanto a navegação, a percepção e o controle simulados permanecem implementados em Python e Pygame.

ANEXO B / INTEGRAÇÃO E AMBIENTE 3D

Modelo do robô e composição da cena

As imagens registram o modelo do Zippy integrado à cena e a organização da visualização tridimensional no editor Unity.

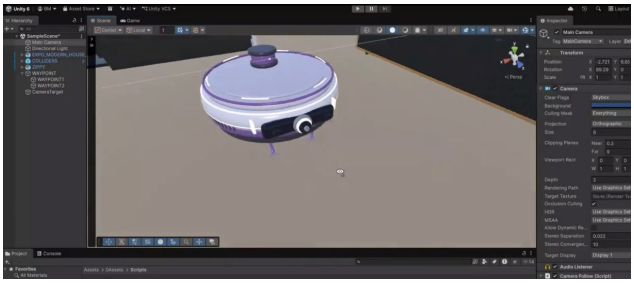


Figura 2. Modelo 3D do Zippy integrado ao cenário no Unity.

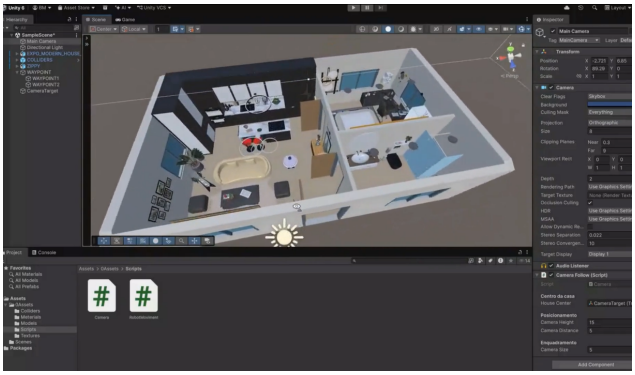


Figura 3. Visualização tridimensional do ambiente residencial.

Integração das ferramentas

Python e Pygame implementam a lógica de navegação, percepção e controle do robô na simulação. O Blender é utilizado na modelagem tridimensional do ambiente residencial e do conceito visual do Zippy. O Unity integra e apresenta esses elementos em um ambiente virtual. Essa divisão permite explorar tanto os aspectos funcionais da robótica quanto sua representação visual em uma residência inteligente.

ANEXO C / TESTES COM MARCADOR HIRO

Testes com o marcador Hiro

Durante o desenvolvimento do projeto, foram realizados testes utilizando o marcador Hiro, uma referência visual empregada em aplicações de Realidade Aumentada (AR). Por meio da câmera de um dispositivo móvel, o marcador pode ser reconhecido e utilizado para posicionar elementos virtuais na tela.

A experimentação teve como objetivo explorar possibilidades de visualização e interação entre elementos físicos e digitais, ampliando os conhecimentos da equipe sobre tecnologias imersivas.

Registro dos testes

Foram realizados testes de reconhecimento do marcador por meio de um celular, verificando sua identificação pela câmera e a visualização do conteúdo virtual.



Figura 4. Marcador Hiro utilizado como referência visual nos testes de reconhecimento por dispositivo móvel.